

Elitismo y Operadores de Selección

September 9, 2025

1 Genetic Algorithm (GA)

- Autor: Rodrigo S. Cortez-Madrigal
- web: <https://roicort.github.io>
- github: <https://github.com/roicort/MetaZoo>
- docs: <https://roicort.github.io/MetaZoo/>

1.0.1 Objetivo

Implementar un Algoritmo Genético (AG) que incorpore elitismo y compare el desempeño de distintos operadores de selección: Ruleta, Muestreo Estocástico Universal (SUS) y Torneo. El propósito es analizar y contrastar sus efectos sobre convergencia, calidad de solución y estabilidad.

1.0.2 Requisitos de implementación

- 1) Elitismo (obligatorio)
 - Integre un esquema de elitismo y especifique cuál utiliza:
 - Mejor individuo (p. ej., 1 élite).
 - Porcentaje de mejores (p. ej., 5–10% de la población).
 - Total (todos los individuos élite pasan sin alteración)

Para esta implementación se utiliza un porcentaje de mejores (configurable por el usuario). [Ver Implementación](#)

- 2) Operadores de selección (obligatorios)
 - Implemente y ejecute el AG con cada uno de los siguientes:
 - Ruleta. [Ver Implementación](#)
 - SUS (Stochastic Universal Sampling). [Ver Implementación](#)
 - Torneo: (indique el tipo de torneo utilizado) [Ver Implementación](#)
 - * Se implementó un tamaño del torneo (k) configurable por el usuario.
 - * Con o sin reemplazo, según se especifique.
 - * Determinístico (el mejor siempre gana) o probabilístico (el mejor gana con cierta probabilidad).
- 3) Línea base (sugerida) Incluya Selección Aleatoria (uniforme) como testigo para ilustrar presión de selección baja y sus implicaciones en convergencia. Implementada como un operador de selección [Ver Implementación](#)

1.0.3 Implementación y Experimentos

```
[1]: from metazoo.bio.evolutionary import GeneticAlgorithm
    from metazoo.bio.evolutionary.operators import selection, mutation, crossover
    from metazoo.gym.mono import Function

    import plotly.io as pio
    #pio.renderers.default = "jupyterlab"
    pio.renderers.default = "svg"

    import pandas as pd
    from rich.progress import track
    import numpy as np
    import plotly.express as px
```

Las funciones de rastrigin, ackley y sphere nos sirve en este caso para comparar los operadores de selección y el elitismo dado que sus minimos se encuentran en el origen (0,0) y son funciones bien conocidas en el ámbito de la optimización.

```
[2]: targets = ['Rastrigin',
    'Ackley',
    'Sphere'
]
```

```
[3]: encoding = 'binary'

    if encoding == 'binary':
        mutation_function = mutation.flip_bit
    else:
        mutation_function = mutation.gaussian
```

Control de variables Ejecutamos para cada entorno de prueba los tres operadores de selección con y sin elitismo. Mantenemos constantes (para cada entorno de prueba) la población, número de generaciones, cruza, mutación y otros parámetros que correspondan, para aislar el efecto de la selección y del elitismo.

```
        crossover_function=crossover.onepoint,
        mutation_function=mutation.flip_bit,
        population_size=100,
        mutation_rate=0.01,
        crossover_rate=0.7,
        precision=3,
```

```
[4]: from functools import partial

    # Torneo de tamaño 5, tipo proporcional, con reemplazo
    tournament = partial(selection.tournament, K=5, type='proportional',
        ↪replace=True)
```

```

selection_operators = [
    selection.uniform,
    selection.roulette,
    selection.stochastic_universal_sampling,
    selection.rank,
    tournament
]

results = []

np.random.seed(42)

for function in track(targets, description="Processing functions..."):
    fitness_function = Function(function, reverse=False)
    for elitism in [None, 0.1, 0.3, 1.0]:
        for selection_ in [selection.uniform, selection.roulette, selection.
↪ stochastic_universal_sampling, selection.rank, selection.tournament]:
            aux_results = []
            aux_best_individual = []
            for _ in range(10): # 10 runs per configuration
                ga = GeneticAlgorithm(
                    fitness_function=fitness_function,
                    crossover_function=crossover.onepoint,
                    mutation_function=mutation_function,
                    selection_function=selection_,
                    population_size=100,
                    mutation_rate=0.01,
                    crossover_rate=0.7,
                    encoding=encoding,
                    bounds=fitness_function.bounds,
                    precision=3,
                    elitism=elitism,
                    minimize=True
                )
                ga.run(generations=100, verbose=False)
                aux_results.append(ga.best_fitness)
                aux_best_individual.append(ga.best_individual)
            mean_best_fitness = sum(aux_results) / len(aux_results)
            best_individual = np.min(np.array(aux_best_individual), axis=0)
            result = {
                "func": function,
                "elitism": elitism,
                "selection": selection_.__name__,
                "best_fitness_mean": mean_best_fitness,
                "best_individual": best_individual
            }

```

```
results.append(result)
```

Output()

1.0.4 Resultados

Juntemos todos los resultados en una tabla para facilitar la comparación.

```
[5]: df = pd.DataFrame(results)
df['best_fitness_mean'] = df['best_fitness_mean'].round(5)
df['best_individual'] = df['best_individual'].apply(lambda x: str([round(i, 5)
    ↪for i in x]) if isinstance(x, list) else x)
df['elitism'] = df['elitism'].fillna(0).astype(str)
```

```
[6]: df
```

```
[6]:
```

	func	elitism	selection	best_fitness_mean \
0	Rastrigin	0.0	uniform	8.19492
1	Rastrigin	0.0	roulette	1.44149
2	Rastrigin	0.0	stochastic_universal_sampling	0.84579
3	Rastrigin	0.0	rank	0.49752
4	Rastrigin	0.0	tournament	0.59759
5	Rastrigin	0.1	uniform	0.00005
6	Rastrigin	0.1	roulette	0.02142
7	Rastrigin	0.1	stochastic_universal_sampling	0.00004
8	Rastrigin	0.1	rank	0.19961
9	Rastrigin	0.1	tournament	0.00004
10	Rastrigin	0.3	uniform	0.00005
11	Rastrigin	0.3	roulette	0.09954
12	Rastrigin	0.3	stochastic_universal_sampling	0.19903
13	Rastrigin	0.3	rank	0.09953
14	Rastrigin	0.3	tournament	0.19903
15	Rastrigin	1.0	uniform	4.55149
16	Rastrigin	1.0	roulette	4.23607
17	Rastrigin	1.0	stochastic_universal_sampling	5.92698
18	Rastrigin	1.0	rank	6.98994
19	Rastrigin	1.0	tournament	6.23908
20	Ackley	0.0	uniform	3.89284
21	Ackley	0.0	roulette	0.11027
22	Ackley	0.0	stochastic_universal_sampling	0.06047
23	Ackley	0.0	rank	0.00123
24	Ackley	0.0	tournament	0.00123
25	Ackley	0.1	uniform	0.00123
26	Ackley	0.1	roulette	0.00123
27	Ackley	0.1	stochastic_universal_sampling	0.00123
28	Ackley	0.1	rank	0.00123

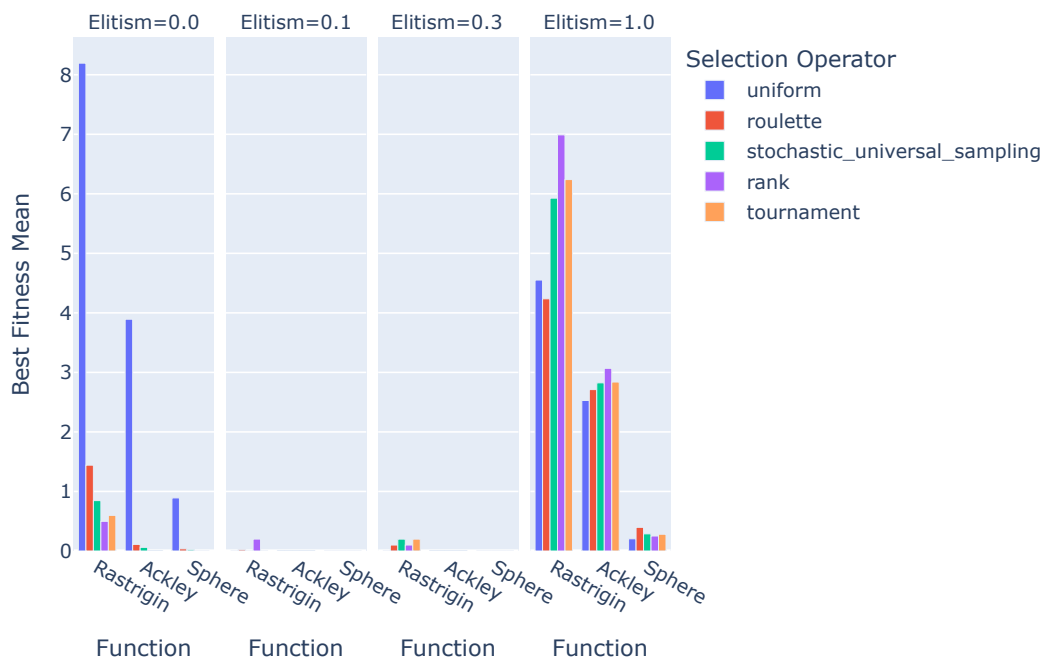
29	Ackley	0.1	tournament	0.00123
30	Ackley	0.3	uniform	0.00123
31	Ackley	0.3	roulette	0.00123
32	Ackley	0.3	stochastic_universal_sampling	0.00123
33	Ackley	0.3	rank	0.00123
34	Ackley	0.3	tournament	0.00123
35	Ackley	1.0	uniform	2.52777
36	Ackley	1.0	roulette	2.71069
37	Ackley	1.0	stochastic_universal_sampling	2.82433
38	Ackley	1.0	rank	3.06809
39	Ackley	1.0	tournament	2.83804
40	Sphere	0.0	uniform	0.89094
41	Sphere	0.0	roulette	0.03568
42	Sphere	0.0	stochastic_universal_sampling	0.02016
43	Sphere	0.0	rank	0.00000
44	Sphere	0.0	tournament	0.00000
45	Sphere	0.1	uniform	0.00000
46	Sphere	0.1	roulette	0.00000
47	Sphere	0.1	stochastic_universal_sampling	0.00000
48	Sphere	0.1	rank	0.00000
49	Sphere	0.1	tournament	0.00000
50	Sphere	0.3	uniform	0.00000
51	Sphere	0.3	roulette	0.00000
52	Sphere	0.3	stochastic_universal_sampling	0.00000
53	Sphere	0.3	rank	0.00000
54	Sphere	0.3	tournament	0.00000
55	Sphere	1.0	uniform	0.20424
56	Sphere	1.0	roulette	0.39689
57	Sphere	1.0	stochastic_universal_sampling	0.28740
58	Sphere	1.0	rank	0.25090
59	Sphere	1.0	tournament	0.28072

	best_individual
0	[-2.076689251052921, -1.9010535310993104]
1	[-1.0128743209424398, -0.9947482146127085]
2	[-0.9947482146127085, -0.9859976805224928]
3	[-0.9947482146127085, -0.00031251907465090767]
4	[-1.0003735579564186, -0.9947482146127085]
5	[-0.0009375572239518348, -0.00031251907465090767]
6	[-0.032814502838308, -0.00031251907465090767]
7	[-0.00031251907465090767, -0.00031251907465090...
8	[-0.00031251907465090767, -1.0003735579564186]
9	[-0.00031251907465090767, -0.00031251907465090...
10	[-0.00031251907465090767, -0.00031251907465090...
11	[-0.00031251907465090767, -0.9953732527620094]
12	[-0.9953732527620094, -0.00031251907465090767]
13	[-0.00031251907465090767, -0.00031251907465090...

14 [-0.9953732527620094, -0.00031251907465090767]
 15 [-1.0591271439907226, -1.929805285967161]
 16 [-2.0660636025148023, -0.9747469938350726]
 17 [-2.0966904718305557, -1.963557346029421]
 18 [-1.9029286455472136, -2.0891900140389428]
 19 [-2.0916901666361474, -1.1303814930110483]
 20 [-1.1002258438625403, -1.0251480192882867]
 21 [-0.04913629982298762, -0.04608435573460312]
 22 [-0.020448025392174607, -0.047915522187633286]
 23 [-0.0003051944088383607, -0.0003051944088383607]
 24 [-0.0003051944088383607, -0.0003051944088383607]
 25 [-0.0003051944088383607, -0.0003051944088383607]
 26 [-0.0003051944088383607, -0.0003051944088383607]
 27 [-0.0003051944088383607, -0.0003051944088383607]
 28 [-0.0003051944088383607, -0.0003051944088383607]
 29 [-0.0003051944088383607, -0.0003051944088383607]
 30 [-0.0003051944088383607, -0.0003051944088383607]
 31 [-0.0003051944088383607, -0.0003051944088383607]
 32 [-0.0003051944088383607, -0.0003051944088383607]
 33 [-0.0003051944088383607, -0.0003051944088383607]
 34 [-0.0003051944088383607, -0.0003051944088383607]
 35 [-0.5203564670695231, -0.3708112067386926]
 36 [-0.7504730513336995, -0.7425379967039003]
 37 [-0.20600622596594054, -0.7468107184276382]
 38 [-1.0538362937190993, -1.0703167917963743]
 39 [-0.3665384850149547, -0.9293169749130197]
 40 [-0.19532442165659525, -2.3473307697002994]
 41 [-0.1084441189037415, -0.5009680766648357]
 42 [-0.0453152658243301, -0.2772044192150398]
 43 [-0.00031251907465090767, -0.00031251907465090...]
 44 [-0.00031251907465090767, -0.00031251907465090...]
 45 [-0.00031251907465090767, -0.00031251907465090...]
 46 [-0.00031251907465090767, -0.00031251907465090...]
 47 [-0.00031251907465090767, -0.00031251907465090...]
 48 [-0.00031251907465090767, -0.00031251907465090...]
 49 [-0.00031251907465090767, -0.00031251907465090...]
 50 [-0.00031251907465090767, -0.00031251907465090...]
 51 [-0.00031251907465090767, -0.00031251907465090...]
 52 [-0.00031251907465090767, -0.00031251907465090...]
 53 [-0.00031251907465090767, -0.00031251907465090...]
 54 [-0.00031251907465090767, -0.00031251907465090...]
 55 [-0.6166001342855401, -0.6134749435390336]
 56 [-0.43471403283891785, -1.0903790514557778]
 57 [-0.7766099005066227, -0.32345724226332173]
 58 [-0.7284819630104371, -0.5234694500396753]
 59 [-0.46471586400537124, -0.36658487456509814]

```
[7]: # Grafiquemos
fig = px.bar(df, x='func', y='best_fitness_mean', color='selection',
             barmode='group',
             facet_col='elitism', title='Best Fitness Mean by Function,
             Selection Operator, and Elitism',
             labels={'best_fitness_mean': 'Best Fitness Mean', 'func':
             'Function', 'selection': 'Selection Operator', 'elitism': 'Elitism'})
fig.update_layout(height=300, width=1200)
fig.show()
```

Best Fitness Mean by Function, Selection Operator, and Elitism



Algunas observaciones interesantes: - La selección uniforme (aleatoria) tiende a tener un desempeño significativamente peor en comparación con los otros métodos de selección. - El elitismo mejora consistentemente el desempeño de todos los métodos de selección, pero su impacto es mayor en la selección por torneo y ruleta. - El elitismo total es la peor estrategia, probablemente porque reduce la diversidad de la población. Mientras que nada de elitismo es mejor que elitismo total, un elitismo moderado (10-30%) parece ofrecer un buen equilibrio entre preservar los mejores individuos y mantener la diversidad genética.

1.0.5 Exploremos las configuraciones de Selección por Torneo

```
[18]: # Experimento adicional: variar K, reemplazo y tipo de torneo

tournament_results = []

np.random.seed(42)

Ks = [2, 3, 5, 10]
replacements = [True, False]
tournament_types = ['standard', 'proportional']
elitism_values = [None, 0.1, 0.3, 1.0]

fitness_fn = Function('Ackley', reverse=False)

for K in track(Ks):
    for replace in replacements:
        for t_type in tournament_types:
            tournament_operator = partial(selection.tournament, K=K,
            ↪type=t_type, replace=replace)
            for elitism in elitism_values:
                run_best_fitness = []
                run_best_individuals = []
                for _ in range(10):
                    ga = GeneticAlgorithm(
                        fitness_function=fitness_fn,
                        crossover_function=crossover.onepoint,
                        mutation_function=mutation_function,
                        selection_function=tournament_operator,
                        population_size=100,
                        mutation_rate=0.01,
                        crossover_rate=0.7,
                        encoding=encoding,
                        bounds=fitness_fn.bounds,
                        precision=3,
                        elitism=elitism,
                        minimize=True
                    )
                    ga.run(generations=100, verbose=False)
                    run_best_fitness.append(ga.best_fitness)
                    run_best_individuals.append(ga.best_individual)
                mean_fit = float(np.mean(run_best_fitness))
                idx_min = int(np.argmin(run_best_fitness))
                best_individual = run_best_individuals[idx_min]
                tournament_results.append({
                    "K": K,
                    "replace": replace,
```



```

        "type": t_type,
        "elitism": elitism,
        "best_fitness_mean": mean_fit,
        "best_individual": best_individual
    })

```

Output()

```

[19]: tournament_df = pd.DataFrame(tournament_results)
tournament_df['elitism'] = tournament_df['elitism'].fillna(0).astype(str)
tournament_df['K'] = tournament_df['K'].astype(str)
tournament_df['best_fitness_mean'] = tournament_df['best_fitness_mean'].round(6)
tournament_df['best_individual'] = tournament_df['best_individual'].apply(
    lambda x: str([round(float(v), 6) for v in x])
)

tournament_df = tournament_df.sort_values(by=["best_fitness_mean"]).
    ↪reset_index(drop=True)
tournament_df

```

```

[19]:
   K  replace      type elitism  best_fitness_mean \
0   2    True  standard     0.0           0.001226
1   3   False  standard     0.1           0.001226
2  10    True  proportional   0.1           0.001226
3   3   False  proportional   0.1           0.001226
4   3   False  proportional   0.3           0.001226
..  ..    ...      ...      ...
59  2    True  proportional   1.0           3.064734
60  5   False  standard     1.0           3.067001
61  3    True  proportional   1.0           3.071563
62  3   False  proportional   1.0           3.074128
63 10   False  standard     1.0           3.294968

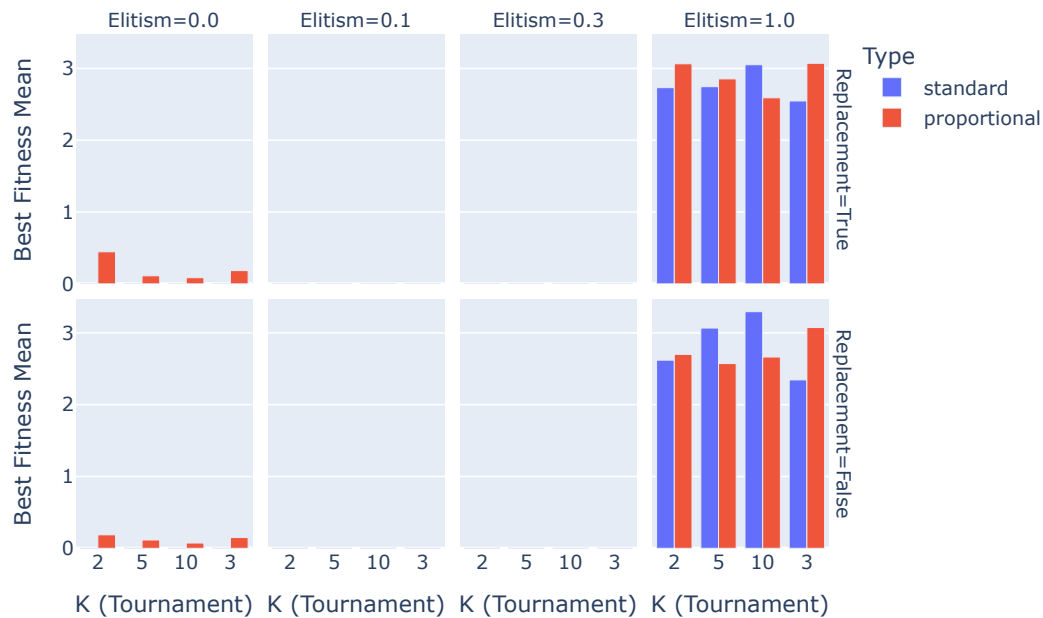
      best_individual
0   [0.000305, -0.000305]
1   [0.000305, 0.000305]
2   [0.000305, 0.000305]
3  [-0.000305, -0.000305]
4   [0.000305, 0.000305]
..  ..
59  [-0.153513, 0.124214]
60  [-0.226149, -0.102851]
61  [0.194409, -0.097357]
62  [-0.031435, -0.020448]
63  [-0.130928, -0.29024]

```

[64 rows x 6 columns]

```
[21]: # Grafiquemos
fig2 = px.bar(tournament_df, x='K', y='best_fitness_mean', color='type',
              ↪barmode='group',
              facet_col='elitism', facet_row='replace',
              title='Best Fitness Mean by Tournament Parameters and Elitism for_
              ↪Ackley',
              labels={'best_fitness_mean': 'Best Fitness Mean', 'K': 'K_
              ↪(Tournament)', 'type': 'Type', 'elitism': 'Elitism', 'replace':_
              ↪'Replacement'})
fig2.update_layout(height=600, width=1000)
fig2.show()
```

Best Fitness Mean by Tournament Parameters and Elitism for Ackley



- Igual aquí podemos observar que el tamaño del torneo (K) tiene un impacto significativo en el desempeño cuando no se utiliza elitismo y sobre todo cuando el tipo de torneo es proporcional.
- Seguimos observando que el elitismo entre el 10-30% ofrece un buen desempeño.
- Cuando no hay elitismo, el torneo proporcional tiene un desempeño menor. Esto podría ser debido a que el torneo proporcional introduce más aleatoriedad en la selección, lo que puede ser perjudicial cuando no se preservan los mejores individuos.
- Usar remplazo parece tener un impacto pequeño en el desempeño, pero mejor que no usarlo.

1.1 Mejores parámetros y Análisis

De acuerdo con los resultados agrupados y ordenados por `best_fitness_mean`, la mejor combinación corresponde a los operadores de selección como torneo o rank junto con un elitismo de entre 0.1 y 0.3.

Esto tiene sentido gracias a que los operadores como torneo y rank ejercen mayor presión en la selección de la población, favoreciendo la supervivencia de los mejores individuos (como en la naturaleza). Esto ayuda a mantener la calidad de la población (con los mejores individuos) a lo largo de las generaciones.

Por otro lado, el elitismo entre 0.1 y 0.3 asegura que los mejores individuos no se pierdan, lo que ayuda a preservar soluciones de alta calidad a lo largo de las generaciones. Por otro lado, si el elitismo es muy alto (1.0) se pierde diversidad y la población se estanca rápidamente, lo que se traduce en perder capacidad de exploración.

2 Criterios

Criterio	W	Esperado	Done
1. Implementación de operadores de selección (Ruleta, SUS, Torneo)	20	Implementa los 3 operadores correctamente; documenta precisión de cada método; para Torneo especifica tipo, k, con/sin reemplazo y resolución de empates; Ruleta maneja aptitudes 0/negativas (normalización/shift) y SUS usa puntos equidistantes.	Torneo seleccionable, con y sin reemplazo y resolución de empates.
2. Elitismo (tipo y parámetros)	20	Integra elitismo y declara explícitamente el tipo (mejor individuo / % mejores / total), el porcentaje/número, si saltan cruza/mutación, y el mecanismo de inserción en la siguiente generación; justifica su elección.	Elitismo con n% mejores como input.
3. Diseño experimental (control de variables + baseline)	15	Mantiene constantes población, generaciones, representación, fitness, cruza, mutación y tasas entre operadores; incluye baseline de Selección Aleatoria; detalla configuración.	En este reporte
5. Visualizaciones (consistencia y legibilidad)	15	Gráficas comparables, líneas de mejor y media, leyendas claras, títulos que incluyen operador y tipo de elitismo; figuras legibles.	En este reporte
6. Resultados y discusión	10	Identifica la mejor combinación (operador + elitismo) y razona causas (presión de selección, diversidad, estancamiento); discute trade-offs (velocidad vs. calidad), efectos de elitismo y del torneo; reconoce limitaciones.	En este reporte
7. Reproducibilidad (código)	10	Código limpio y modular; requisitos, comandos de ejecución, parámetros, es posible replicar todas las condiciones; estructura de carpetas clara.	En repositorio

Criterio	W	Esperado	Done
8. Informe (PDF) claridad y estructura	10	Breve y claro: problema, AG (representación/fitness/cruza/mutación), operadores y tipo de torneo, elitismo (tipo y parámetros), setup experimental, resultados, gráficas y tabla resumen, conclusiones; redacción formal.	En este reporte